Heap Sort

AlgoExpert · Hard · Sorting

The problem

- Build a max-heap in place then repeatedly move the max to the end.

Intuition

- Build a max-heap in place, then repeatedly move the maximum to the end and restore the heap.

Approach

- 1. Heapify from the last parent down
- 2. Swap root with the last element
- 3. Sift down within the shrinking heap

Java solution

```java
public class HeapSort {
    public static int[] heapSort(int[] a) {
        for (int i = a.length / 2 - 1; i >= 0; i--) sift(a, i, a.length);
        for (int end = a.length - 1; end > 0; end--) { int t = a[0]; a[0] = a[end]; a[end] = t; sift(a, 0, end); }
        return a;
    }

    private static void sift(int[] a, int i, int n) {
        while (2 * i + 1 < n) {
            int c = 2 * i + 1;
            if (c + 1 < n && a[c + 1] > a[c]) c++;
            if (a[i] >= a[c]) return;
            int t = a[i]; a[i] = a[c]; a[c] = t;
            i = c;
        }
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(1)

Recap

- Build a max-heap in place, then repeatedly move the maximum to the end and restore the heap.
- Complexity: O(n log n)